Narwhal Labs - Technical Assessment

Context

A Kubernetes experienced a prolonged outage after an AWS Aurora failover, which took the writer offline for ~40 seconds and caused pod failure for 8 minutes with *CrashLoopBackOff.* The application is a small Python app deployed with a YAML manifest.

This report aims to implement reliability improvements to prevent incident recurrence.

Discovery

Database Health Check

Run the application locally via Python to review health check behaviour.

```
DB_UP=false python3 app.py
curl http://localhost:8080/healthz
```

Alternatively, use Docker for a closer approximation to a Production Kubernetes cluster.

```
docker build . -t python-app
docker run python-app
docker exec <id> curl -s http://localhost:8080/healthz
```

This uses a simple Dockerfile:

```
FROM python:3.15
ENV DB_UP=true
EXPOSE 8080
ADD app.py .
ENTRYPOINT ["python", "app.py"]
```

Liveness & Readiness Probes

A *readiness probe* is a diagnostic check to determine whether a container is ready to accept network traffic. A *liveness probe* checks a service is running normally and kills/restarts unhealthy containers.

- Both probes use the same endpoint at `/healthz` which checks database availability
- Both probes have a delay of 2s, frequency of 5s, timeout of 2s and threshold of 3
- The health check must fail 3 times over a total of 15s to trigger a restart
- The health check must be successful within 2s of the pod starting

Root Cause Analysis

The primary cause of this incident was a misconfiguration of Kubernetes readiness and liveness probes. These both use the same endpoint `/healthz` which opens a database connection and runs `SELECT 1`. A readiness probe is designed to route traffic away from unhealthy pods; liveness should not restart pods due to a temporary database blip. Additionally, the 2-second startup delay provides little margin for application warm-up.

Timeline

1. The Aurora database becomes unavailable after a failover
2. The pod liveness and readiness probes at `/healthz` fail or timeout
3. After 3 consecutive failures over 15 seconds, Kubernetes restarts the container
4. The new container fails readiness checks and enters `CrashLoopBackOff` with exponential delays of 10, 20 and 40 seconds
5. The Aurora database recovers after 40 seconds
6. The container fails to restart - possible reasons include slow server warm-up, resource exhaustion, DNS caching, connection limits
7. The container re-enters `CrashLoopBackOff` with larger delays of 80, 160 or 300 seconds
8. The container recovers after startup and the readiness probe is successful

<table>
<tr><th>Reason</th><th>Evidence</th><th>Action</th><th>Scope</th></tr>
<tr><td>The liveness and readiness probe use the same endpoint</td><td><pre>livenessProbe:
  httpGet:
  path: /healthz
readinessProbe:
  httpGet:
  path: /healthz</pre></td><td>Implement a separate readiness probe endpoint that does not rely on database availability</td><td>Immediate</td></tr>
<tr><td>The liveness and readiness probes have a short delay</td><td><pre>readinessProbe:
  httpGet:
    path: /healthz
    port: http
  initialDelaySeconds: 2</pre></td><td>Increase the initial delay or implement a startup probe</td><td>Immediate</td></tr>
<tr><td>There is no startup probe</td><td>N/A</td><td>Implement a startup probe</td><td>Immediate</td></tr>
<tr><td>The deployment has a single replica and no scaling</td><td><pre>spec:
  replicas: 1</pre></td><td>Implement HPA with multiple replicas</td><td>Future</td></tr>
<tr><td>The pod does not have resource constraints</td><td>N/A</td><td>Add resource constraints for CPU and memory</td><td>Future</td></tr>
</table>

Implementation

Implement a separate readiness probe endpoint that does not rely on database availability

A readiness probe is designed to check if an application is responsive, and should not rely on database availability. It should not use the same endpoint as the liveness probe.

Add a new readiness endpoint in *app.py:*

```
if self.path == "/livez":
    self._respond(200, "ok\n")
```

Use the new readiness endpoint in *deployment.yaml:*

```
readinessProbe:
  httpGet:
    path: /livez
    port: http
```

Increase the initial delay or implement a startup probe

A startup probe ensures the application has enough time to load voice models before it can serve traffic.

```
startupProbe:
    httpGet:
      path: /livez
      port: http
    periodSeconds: 5
    failureThreshold: 3
```

Implement HPA with multiple replicas

The application will be significantly more reliable with multiple replicas. A HPA (Horizontal Pod Autoscaler) will scale the deployment based on resource utilisation.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: inbox-api
  namespace: inbox
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: inbox-api
  minReplicas: 2
  maxReplicas: 3
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 70
```

Add resource constraints for CPU and memory

Resource constraints prevent containers from overcommitting. This allows the cluster scheduler to balance workloads more efficiently and reduces cloud costs.

```yaml
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "200m"
    memory: "256Mi"
```

Post-Verification

Run a final review and validation check via CoPilot; *"Check this Kubernetes manifest and Python app and confirm whether the code is valid. Check the design against best practices. Are the health probes, resource constraints and HPA reliable?"*

Test the application locally:

```
DB_UP=false python3 app.py
curl http://localhost:8080/healthz
```

Publish the application to Github:

```
git init
git add .
git status
git commit -m "Initial commit"
git remote add NarwhalLabsTechnical https://github.com/emiliaszynkowska/NarwhalLabsTechnical
git push
```

emiliaszynkowska/NarwhalLabsTechnical Explain